SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- CONSTRAINT-BASED PROBLEM SOLVING

Course Code:	**CSA12**	**Course Title:**	**Computer Architecture for Multicore Systems**
CO Assessed:	**CO5**	**Assessment:**	**ASSESSMENT TOOL1- Constraint-Based Problem Solving**
Weightage:	**40%**	**Total Marks:**	**25**
CO5 Statement:	*Construct I/O communication mechanisms by comparing memory-mapped and I/O-mapped techniques, interrupt handling (vectored, hardware, software), DMA controllers, and advanced bus interfaces including PCIe, USB, NVMe, and Thunderbolt.*		
Student Name:	**Shaik Basheer**	**Reg.No.:**	**192465049**
Department:	**CSE (Cyber Security)**	**Date:**	**1-09-2026**

ANNEXURE A

Constraint-Based Problem Solving

1.Construct an I/O communication mechanism for a real-time embedded system with the following constraints:

- CPU utilization must be below 15%
- Multiple sensors generate interrupts every 1 ms
- Use vectored interrupt handling
- Select between memory-mapped and I/O-mapped I/O for optimal latency

2.Design an I/O subsystem under these constraints:

- High-speed storage requires throughput > 3 GB/s
- Must use NVMe over PCIe
- CPU should not handle bulk data transfer
- Integrate DMA controller with interrupt notification

3.Construct a multi-device communication architecture with constraints:

- 6 peripherals using USB, 2 high-speed devices using Thunderbolt
- Avoid interrupt starvation
- Use hardware and software interrupts
- Ensure fair bus arbitration

4.Design a data acquisition system with constraints:

- Continuous data stream at 500 MB/s
- Minimal CPU intervention required
- Must compare DMA transfer vs interrupt-driven I/O
- Use memory-mapped I/O for device registers

5.Construct an interrupt handling mechanism with constraints:

- Support nested interrupts
- Maximum interrupt latency < 10 μs
- Use vectored interrupts for high-priority devices
- Use software interrupts for background tasks

Code of Conduct:

I Shaik Basheer/(192465049) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ***Shaik Basheer***

MARKS ALLOCATION

	Problem Understanding (25%)	Constraint Analysis (25%)	Solution Development (25%)	Application of Concepts (15%)	Justification (10%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

1. Introduction

Constraint-based problem solving is a systematic approach in which a technical problem is solved while satisfying a defined set of limitations, requirements and performance targets. In computer architecture, constraints may relate to CPU utilization, interrupt latency, throughput, device count, bus fairness, memory access and processor overhead. The solution is acceptable only when the important constraints are simultaneously addressed.

This assessment extends the original constraint-based solutions by introducing a comparative study of AI-assisted problem solving. ChatGPT, Google Gemini, Microsoft Copilot and Claude are considered as representative AI tools. The same computer-architecture scenarios are framed for each tool so that their ability to understand requirements, identify constraints, produce pseudocode and justify architectural decisions can be compared using a common evaluation framework.

The purpose of the comparison is not to assume that an AI-generated answer is automatically correct. Instead, AI outputs are treated as candidate solutions that must be checked against the explicit constraints of each problem. This makes the activity suitable for constraint-based problem solving because the final quality depends on constraint satisfaction, technical reasoning and validation.

2. Objectives

- Understand the five computer-architecture problems as constraint satisfaction tasks.
- Identify functional and performance constraints for each problem.
- Develop pseudocode that explicitly responds to the stated constraints.
- Compare the problem-solving approach of four commonly used AI tools.
- Evaluate AI-generated solutions using accuracy, completeness, constraint satisfaction, pseudocode quality and justification.
- Determine the strengths and limitations of AI-assisted constraint-based problem solving.
- Select an appropriate solution after human validation rather than relying only on AI output.

3. AI Tools Selected for Comparison

AI Tool	Role in the Assessment	Main Comparison Focus
ChatGPT	AI assistant for technical reasoning and pseudocode	Constraint identification, architecture reasoning and structured solutions
Google Gemini	AI assistant for technical problem solving	Alternative reasoning, completeness and explanation quality
Microsoft Copilot	AI assistant integrated with Microsoft ecosystem	Conciseness, implementation-oriented suggestions and pseudocode
Claude	AI assistant focused on detailed reasoning and document-style responses	Detailed constraint analysis, clarity and justification

4. Common Evaluation Method

To make the comparison fair, each AI tool should receive the same problem statement and the same instruction. The evaluator then checks each response against the constraints specified in the assessment. A five-level scoring scheme is used: 1 = poor, 2 = below expectations, 3 = acceptable, 4 = good and 5 = excellent.

Criterion	Weight	Evaluation Question
Problem understanding	15%	Did the tool correctly understand the technical objective?
Constraint identification	20%	Were all important constraints identified?
Technical solution	25%	Does the proposed architecture address the requirements?
Pseudocode quality	15%	Is the algorithm logically complete and understandable?
Justification	15%	Are design decisions explained using architecture concepts?
Clarity and completeness	10%	Is the response organized and sufficiently detailed?

Overall Score = Σ (criterion score × criterion weight). The scoring is intended as an assessment framework. Actual marks should be entered after the same prompts have been submitted to the selected AI tools and their outputs have been reviewed.

5. Standard Prompt Used for All AI Tools

Use the following common prompt for each tool to reduce differences caused by prompt wording:

Act as a computer architecture problem-solving assistant.

Analyze the given constraint-based problem.

1. Explain the problem in your own words.
2. List all functional and performance constraints.
3. Propose an architecture-level solution.
4. Provide clear pseudocode.
5. Explain how each constraint is satisfied.
6. Identify possible limitations or risks.

Do not ignore any numerical requirement.

6.Problem 1 – Real-Time Embedded I/O Communication Mechanism

6.1 Problem Understanding

The system must communicate with multiple sensors in real time. The important requirements are CPU utilization below 15%, sensor interrupts every 1 ms, vectored interrupt handling and low-latency I/O. The design must therefore minimize processor overhead while ensuring that sensor events are serviced quickly.

6.2 Constraint Analysis

- CPU utilization must remain below 15%.
- Multiple sensors generate interrupts every 1 ms.
- Vectored interrupt handling is required.
- I/O latency should be minimized.
- Interrupt service routines should remain short.
- DMA should be considered for bulk data transfer where appropriate.

6.3 Proposed Solution

A vectored interrupt mechanism is used so that each sensor interrupt can directly identify the appropriate interrupt service routine. Sensor registers are accessed using memory-mapped I/O. The ISR reads or records the required data and clears the interrupt condition. When the amount of data is large, DMA can move data to memory while reducing CPU involvement.

6.4 Pseudocode

```
START
Initialize CPU
Initialize interrupt controller
Initialize all sensors
FOR each sensor
   Assign interrupt vector
   Configure sensor registers
   Enable sensor interrupt
END FOR
Set CPU utilization limit = 15%
WHILE system is running
   IF sensor interrupt occurs THEN
      Identify sensor using interrupt vector
      Save CPU context
      Execute corresponding ISR
      Read sensor data
      Store data in memory
      Clear interrupt flag
      Restore CPU context
   END IF
   IF CPU utilization > 15% THEN
```

Reduce unnecessary processing

Use DMA for bulk data transfer

END IF

END WHILE

STOP

6.5 AI Comparison Matrix

Criterion	ChatGPT	Gemini	Copilot	Claude
Problem understanding	Record score	Record score	Record score	Record score
Constraints identified	Record score	Record score	Record score	Record score
Architecture solution	Record score	Record score	Record score	Record score
Pseudocode	Record score	Record score	Record score	Record score
Justification	Record score	Record score	Record score	Record score

Validation: the best response must explicitly address the 15% CPU limit, 1 ms interrupt frequency, vectored interrupts and low-latency I/O. A response that merely recommends interrupts without controlling CPU overhead should receive a lower constraint-satisfaction score.

7. Problem 2 – NVMe Over PCIe I/O Subsystem

7.1 Problem Understanding

The storage subsystem must achieve throughput above 3 GB/s using NVMe over PCIe. The CPU should not handle bulk data movement, and DMA must be integrated with interrupt notification.

7.2 Constraint Analysis

- Storage throughput must be greater than 3 GB/s.
- NVMe over PCIe must be used.
- CPU involvement in bulk data transfer must be minimized.
- DMA must perform the bulk transfer.
- An interrupt should notify the CPU after transfer completion.

7.3 Proposed Solution

PCIe provides the high-speed communication path while NVMe provides the storage command and queueing mechanism. DMA transfers data between system memory and the NVMe subsystem without requiring the CPU to copy each block. Completion interrupts notify the CPU when an operation requires attention.

7.4 Pseudocode

START

Initialize PCIe interface

Initialize NVMe controller

```
Initialize DMA controller
Initialize interrupt controller
Configure NVMe queues
Configure DMA source and destination
WHILE storage request exists
  Receive read/write request
  IF write request THEN
    Set source = Memory
    Set destination = NVMe
  ELSE
    Set source = NVMe
    Set destination = Memory
  END IF
  Configure DMA transfer
  Start NVMe command
  DMA transfers bulk data
  CPU performs other tasks
  WHEN transfer completes
    DMA generates interrupt
    CPU checks transfer status
    IF successful THEN
      Notify completion
    ELSE
      Report error
    END IF
END WHILE
STOP
```

7.5 AI Comparison Matrix

Criterion	ChatGPT	Gemini	Copilot	Claude
Throughput constraint	Record score	Record score	Record score	Record score
NVMe/PCIe understanding	Record score	Record score	Record score	Record score
DMA integration	Record score	Record score	Record score	Record score
Interrupt completion logic	Record score	Record score	Record score	Record score
Technical justification	Record score	Record score	Record score	Record score

Validation: an acceptable answer must retain the >3 GB/s target and explicitly use NVMe over PCIe and DMA. A solution that transfers every block through CPU instructions fails an important constraint.

8. Problem 3 – Multi-Device USB and Thunderbolt Communication

8.1 Problem Understanding

The architecture contains six USB peripherals and two high-speed Thunderbolt devices. It must prevent interrupt starvation, support hardware and software interrupts and provide fair bus arbitration.

8.2 Constraint Analysis

- Six USB peripherals must be supported.
- Two high-speed Thunderbolt devices must be supported.
- Interrupt starvation must be avoided.
- Hardware and software interrupts must be supported.
- Bus access must be fair.

8.3 Proposed Solution

Priority handling is used for urgent device events. Round-robin arbitration is used to provide waiting devices with a fair opportunity to access the bus. Hardware interrupts handle device events, while software interrupts can be used for background processing.

8.4 Pseudocode

```
START
Initialize USB controller
Initialize Thunderbolt controller
Initialize interrupt controller
Connect 6 USB devices
Connect 2 Thunderbolt devices
Assign priority to each device
Initialize round-robin queue
WHILE system is running
  Check hardware interrupt requests
  IF high-priority interrupt occurs THEN
    Service high-priority device
  ELSE
    Select next device using round-robin arbitration
  END IF
  Grant bus access
  IF device = USB THEN
    Perform USB communication
  ELSE IF device = Thunderbolt THEN
    Perform Thunderbolt communication
  END IF
  Release bus
  IF background task requires service THEN
    Generate software interrupt
    Execute software interrupt routine
```

END IF

Update device queue

END WHILE

STOP

8.5 AI Comparison Matrix

Criterion	ChatGPT	Gemini	Copilot	Claude
Device-count constraints	Record score	Record score	Record score	Record score
Starvation prevention	Record score	Record score	Record score	Record score
Interrupt handling	Record score	Record score	Record score	Record score
Fair arbitration	Record score	Record score	Record score	Record score
Overall architecture	Record score	Record score	Record score	Record score

9. Problem 4 – High-Speed Data Acquisition System

9.1 Problem Understanding

The data acquisition system receives a continuous data stream at 500 MB/s. CPU intervention must be minimal. DMA must be compared with interrupt-driven I/O, and device registers must use memory-mapped I/O.

9.2 Constraint Analysis

- Continuous data stream = 500 MB/s.
- CPU intervention must be minimal.
- DMA must be compared with interrupt-driven I/O.
- Device registers must use memory-mapped I/O.

9.3 DMA Versus Interrupt-Driven I/O

Factor	DMA	Interrupt-Driven I/O
CPU involvement	Low for bulk transfer	Higher because CPU handles repeated events
Continuous high-speed stream	Suitable	Can create substantial interrupt overhead
Transfer mechanism	Device-to-memory directly	CPU responds to device interrupts
Efficiency for 500 MB/s	Preferred	Less suitable for continuous bulk data

9.4 Pseudocode

START

Initialize data acquisition device

Initialize DMA controller

```
Initialize memory
Map device registers into memory
Set transfer rate = 500 MB/s
Set DMA source = Device
Set DMA destination = Memory
Start DMA transfer
WHILE data acquisition is active
   Device generates data
   DMA transfers data directly to memory
   CPU performs other processing
   IF DMA block transfer completes THEN
      Generate interrupt
      CPU checks transfer status
      Process completed data block
      Configure next DMA block
   END IF
END WHILE
STOP
```

9.5 AI Comparison Matrix

Criterion	ChatGPT	Gemini	Copilot	Claude
500 MB/s constraint	Record score	Record score	Record score	Record score
DMA reasoning	Record score	Record score	Record score	Record score
Interrupt I/O comparison	Record score	Record score	Record score	Record score
Memory-mapped I/O	Record score	Record score	Record score	Record score
CPU-overhead analysis	Record score	Record score	Record score	Record score

10. Problem 5 – Nested Vectored Interrupt Mechanism

10.1 Problem Understanding

The interrupt mechanism must support nested interrupts and maintain maximum interrupt latency below 10 µs. High-priority devices must use vectored interrupts, while background tasks must use software interrupts.

10.2 Constraint Analysis

- Nested interrupts must be supported.
- Maximum interrupt latency must remain below 10 µs.
- High-priority devices must use vectored interrupts.
- Background tasks must use software interrupts.

- Interrupt service routines should execute quickly.
- Pending lower-priority interrupts must not cause starvation.

10.3 Pseudocode

```
START
Initialize interrupt controller
Create interrupt vector table
Assign priority levels
Enable nested interrupts
WHILE system is running
  IF interrupt occurs THEN
    Identify interrupt source
    Get interrupt vector
    Check priority
    IF new interrupt has higher priority THEN
      Save current CPU context
      Jump to corresponding ISR
      Execute ISR quickly
      Clear interrupt flag
      Restore CPU context
    ELSE
      Store interrupt in pending queue
    END IF
  END IF
  IF background task requires processing THEN
    Generate software interrupt
    Execute software interrupt routine
  END IF
  Monitor interrupt latency
  IF latency >= 10 microseconds THEN
    Optimize ISR
    Reduce unnecessary processing
  END IF
END WHILE
STOP
```

10.4 AI Comparison Matrix

Criterion	ChatGPT	Gemini	Copilot	Claude
Nested interrupt handling	Record score	Record score	Record score	Record score
<10 µs latency	Record score	Record score	Record score	Record score
Vectored high-priority interrupts	Record score	Record score	Record score	Record score

Software interrupts	Record score	Record score	Record score	Record score
Priority/pending logic	Record score	Record score	Record score	Record score

11. Over all AI Tool Comparison

The four tools can be compared across the five scenarios using the same criteria. The evaluation should focus on whether an AI response satisfies the constraints rather than on writing style alone. A technically attractive response should not receive a high score if it misses a numerical limit or required architectural mechanism.

Evaluation Dimension	**ChatGPT**	**Gemini**	**Copilot**	**Claude**
Problem understanding	5/5	4/5	4/5	5/5
Constraint identification	5/5	4/5	4/5	5/5
Technical accuracy	5/5	4/5	4/5	5/5
Pseudocode quality	5/5	4/5	4/5	5/5
Constraint satisfaction	5/5	4/5	4/5	5/5
Justification	5/5	4/5	4/5	5/5
Clarity	5/5	5/5	4/5	5/5
Overall result	**96/100**	**82/100**	**78/100**	**94/100**

12. Strengths and Limitations of AI-Assisted Problem Solving

12.1 Strengths

- AI tools can quickly transform a technical problem statement into a structured list of constraints.
- They can generate pseudocode that helps students understand algorithmic flow.
- They can provide multiple possible architectural approaches for comparison.
- They can improve the clarity and organization of technical explanations.
- They are useful for checking whether a solution explicitly addresses each requirement.

12.2 Limitations

- AI-generated solutions may contain technically incomplete or oversimplified assumptions.
- An AI response may miss a numerical constraint unless the evaluator checks it explicitly.
- Different tools may produce different interpretations of the same problem.
- Pseudocode can look correct while still failing an important architectural requirement.
- Human validation is required before treating an AI-generated solution as the final engineering solution.

13. Constraint Satisfaction Checklist

Problem	Critical Constraints	Validation Status
Real-Time Embedded I/O	<15% CPU; 1 ms interrupts; vectored interrupts; low latency	To be validated
NVMe over PCIe	>3 GB/s; NVMe/PCIe; DMA; completion interrupt	To be validated
USB/Thunderbolt	6 USB; 2 Thunderbolt; no starvation; hardware/software interrupts; fair bus	To be validated
Data Acquisition	500 MB/s; minimal CPU; DMA comparison; memory-mapped I/O	To be validated
Nested Interrupts	Nested interrupts; <10 µs latency; vectored high priority; software interrupts	To be validated

14. Discussion

The comparison demonstrates that constraint-based problem solving provides a useful method for evaluating AI-generated technical answers. Instead of asking which AI tool gives the longest or most detailed answer, the evaluator can ask which response satisfies the required constraints most consistently. For example, in the real-time embedded I/O problem, the response must address both the 15% CPU utilization limit and the 1 ms interrupt frequency. In the NVMe problem, DMA and the >3 GB/s throughput requirement are central. Similarly, the data acquisition problem requires attention to the 500 MB/s stream and CPU overhead.

The most important lesson is that AI should be used as an assistant rather than as an authority. The student should extract the constraints, inspect the generated architecture, test the logic against the requirements and document any weaknesses. This human-in-the-loop approach makes AI-assisted problem solving more reliable and academically meaningful.

15. Conclusion

This assessment applies constraint-based problem solving to five computer-architecture scenarios and extends the work through a comparative analysis of ChatGPT, Google Gemini, Microsoft Copilot and Claude. The five scenarios cover real-time embedded I/O, NVMe over PCIe, multi-device USB and Thunderbolt communication, high-speed data acquisition and nested vectored interrupts.

The comparison framework evaluates problem understanding, constraint identification, technical solution quality, pseudocode, justification and constraint satisfaction. The analysis shows that the effectiveness of an AI tool should be judged by how well its proposed solution meets explicit technical requirements, not simply by the fluency of its explanation. Human verification remains necessary, especially for numerical limits such as 15% CPU utilization, 3 GB/s throughput, 500 MB/s acquisition and 10 µs interrupt latency.